

RELATÓRIO TÉCNICO: DESENVOLVIMENTO DA APLICAÇÃO WEB – PIZZARIA JESUS

Alunos: Eduardo Diniz dos Santos - 25153454-2 (NA), Gabriel Grimm Martins - 25033070-2 (NA), João Pedro Hulchak Kazmierczak - 25141620-2 (NA), Hiuri Luciano dos Santos - 25208360-2 (NA).

1. INTRODUÇÃO

Este documento aborda tanto o processo de desenvolvimento quanto a base técnica da aplicação web Pizzaria Jesus, que se apresenta como um e-commerce/cardápio digital centrado na experiência de escolha e aquisição de produtos alimentícios. A proposta principal do projeto é desenvolver uma plataforma que seja moderna, intuitiva e de fácil navegação para o usuário final.

Dentre as funcionalidades mais relevantes que foram implementadas, estão a renderização dinâmica de um cardápio de pizzas a partir de dados estruturados, um sistema de gerenciamento de carrinho de compras em tempo real e uma interface de finalização de pedido (checkout) que valida e persiste dados localmente.

2. DESENVOLVIMENTO

2.1 Tecnologias Utilizadas

A arquitetura da aplicação foi elaborada com o auxílio do atual ecossistema de desenvolvimento front-end, que inclui:

- HTML5 e JavaScript (ES6+): Linguagens essenciais para a organização lógica e semântica de toda a aplicação.
- React 18: Biblioteca fundamental para construir a interface baseada em componentes, otimização na manipulação do DOM Virtual e controle de estados.
- Tailwind CSS: Framework utilitário para estilização rápida, responsiva e fundamentada em Design Systems.
- Vite: Uma ferramenta de build e bundling rápida que serve como servidor de desenvolvimento local.
- React Router Dom (v6): Biblioteca encarregada do roteamento declarativo e da

navegação entre páginas.

2.2 Estrutura HTML e Semântica

A marcação da interface priorizou o uso de HTML5 semântico para garantir acessibilidade e SEO adequados. Foram empregadas tags estruturais específicas para segmentar o conteúdo de forma lógica:

- `<header>` e `<nav>` para o cabeçalho e os links de navegação global.
- `<main>` como container do conteúdo principal de cada rota.
- `<section>` e `<article>` para delimitar as áreas de exibição do cardápio e o encapsulamento individual de cada produto (PizzaCard).
- `<footer>` para as informações institucionais localizadas no rodapé da aplicação.

2.3 Estratégias de Layout e Responsividade

O design visual da plataforma adota os conceitos fundamentais de posicionamento e fluxo de elementos:

- **Box Model:** Controle rigoroso de espaçamentos internos (*padding*), margens externas (*margin*) e bordas para garantir o equilíbrio visual.
- **Flexbox:** Utilizado para o alinhamento unidimensional de componentes, como a distribuição de links na barra de navegação superior e o alinhamento de textos e botões dentro dos cartões.
- **Grid Layout:** Aplicado na página do cardápio para dispor os produtos em uma grade bidimensional organizada, adaptando o número de colunas conforme o tamanho da tela.
- **Responsividade:** Garantida nativamente por meio de utilitários de *Media Queries* do Tailwind CSS (como as diretivas `sm:`, `md:` e `lg:`), permitindo uma transição fluida do layout entre dispositivos móveis, tablets e monitores desktop.

2.4 Lógica de Programação e Interatividade

A inteligência do sistema foi construída combinando recursos nativos da linguagem JavaScript e APIs do ecossistema Web:

- **Validação de Formulários:** O formulário de *checkout* utiliza lógica em JavaScript para auditar os campos obrigatórios inseridos pelo usuário antes do processamento do pedido, mitigando envios incompletos ou dados vazios.

- **Manipulação de Eventos e Listeners:** Captura ativa de ações do usuário por meio de ouvintes de eventos (como onClick nos botões de adição ao carrinho e onChange na captura de dados digitados).
- **Armazenamento Local (LocalStorage):** Persistência do estado do carrinho de compras em memória local do navegador, garantindo que os itens selecionados não sejam perdidos caso o usuário recarregue a página por acidente.
- **Requisições Externas (Fetch API e JSON):** O cardápio de pizzas é alimentado de forma assíncrona por meio de uma requisição fetch que consome um arquivo estruturado no formato menu.json, simulando o consumo real de uma API REST de backend.

2.5 Estruturação em React

A aplicação foi completamente modelada sob a arquitetura do React:

- **Componentização:** Divisão da interface em partes isoladas e reutilizáveis, segmentadas em componentes globais (ex: Header, Footer, PizzaCard) e páginas exclusivas (Home, Menu, Cart, Checkout).
- **Propriedades (Props):** Fluxo de dados unidirecional onde o componente pai (Menu) transmite dinamicamente as informações individuais de cada pizza para o componente filho (PizzaCard) realizar a renderização visual.
- **Gerenciamento de Estado (useState):** Utilizado para gerenciar dados reativos e mutáveis da interface, como a lista dinâmica de pizzas carregadas e o controle de itens adicionados ao carrinho de compras.
- **Gerenciamento de Efeitos (useEffect):** Hook encarregado de monitorar os ciclos de vida dos componentes, disparando a requisição assíncrona do arquivo JSON no momento da montagem inicial da página e sincronizando o estado atualizado do carrinho com o LocalStorage.
- **Roteamento Dinâmico (React Router):** Implementação de uma arquitetura *Single Page Application* (SPA), onde o cliente navega instantaneamente pelas URLs do sistema (Início, Cardápio, Carrinho) por meio do mapeamento de rotas, alterando o DOM Virtual sem provocar o recarregamento total da página web.

3. DESAFIOS ENCONTRADOS E SOLUÇÕES

Durante a execução prática do projeto, surgiram desafios de infraestrutura de ambiente e de compilação de código que exigiram diagnóstico técnico detalhado:

1. **Bloqueio de Segurança no Diretório Local (Windows Defender):** No estágio inicial de instalação de dependências, o instalador do Node (npm install) falhou devido a uma interferência do recurso "Acesso Controlado a Pastas" da Segurança do Windows, que bloqueou a execução do node.exe no diretório padrão de Documentos.
 - *Solução:* O projeto foi reestruturado e movido para uma pasta diretamente localizada na raiz do sistema operacional (C:\GitHub\PizzariaJesus) e as políticas locais de segurança foram liberadas, permitindo a livre manipulação do diretório de pacotes node_modules.
2. **Configuração Manual das Dependências do Vite:** Por não utilizar um utilitário automático de scaffold padrão do zero, a aplicação carecia dos arquivos base de configuração na raiz do projeto, impossibilitando a leitura inicial do compilador.
 - *Solução:* Foi necessária a estruturação manual detalhada do arquivo de manifesto package.json, especificando scripts de execução, dependências de produção (React e Router) e ferramentas de desenvolvimento (Tailwind CSS, PostCSS e Vite), sanando o erro e preparando o ambiente local de execução.
3. **Resolução de Caminhos de Importação e Case Sensitivity:** O servidor de desenvolvimento acusou o erro [plugin:vite:import-analysis] na página Menu.jsx, decorrente de uma falha na localização e análise do caminho relativo direcionado ao componente de exibição dos cards.
 - *Solução:* Realizou-se uma auditoria no código fonte para ajustar o caminho absoluto do arquivo e incluir explicitamente a extensão .jsx no comando de importação, respeitando a sensibilidade estrita do Vite quanto ao uso de letras maiúsculas e minúsculas no mapeamento do sistema de arquivos.

4. CONCLUSÃO

O desenvolvimento da **Pizzaria Jesus** consolidou os conhecimentos práticos exigidos no escopo do desenvolvimento front-end moderno. A transição de um paradigma imperativo tradicional para a mentalidade declarativa e orientada a componentes do React permitiu compreender com clareza os ganhos de performance proporcionados pelo gerenciamento automatizado do DOM Virtual e pelo controle de ciclos de vida de estados reativos.

Como sugestões de melhoria contínua e escalabilidade para o sistema atual, propõe-se:

1. **Integração com Backend Real:** Substituição do arquivo local menu.json por uma API REST estruturada em Node.js ou C# conectada a um banco de dados relacional, permitindo a persistência definitiva dos pedidos realizados.
2. **Painel de Monitoramento da Cozinha:** Criação de uma rota restrita com comunicação via WebSockets (como Socket.io), permitindo que o estabelecimento receba e atualize o status de produção das pizzas em tempo real.
3. **Gateway de Pagamento Integrado:** Implementação de uma API de pagamentos (como Stripe ou Mercado Pago) na tela de finalização da compra para a validação financeira real das transações simuladas.

5. CAPTURAS DE TELA

1: Interface Principal do Cardápio Digital (Responsivo)

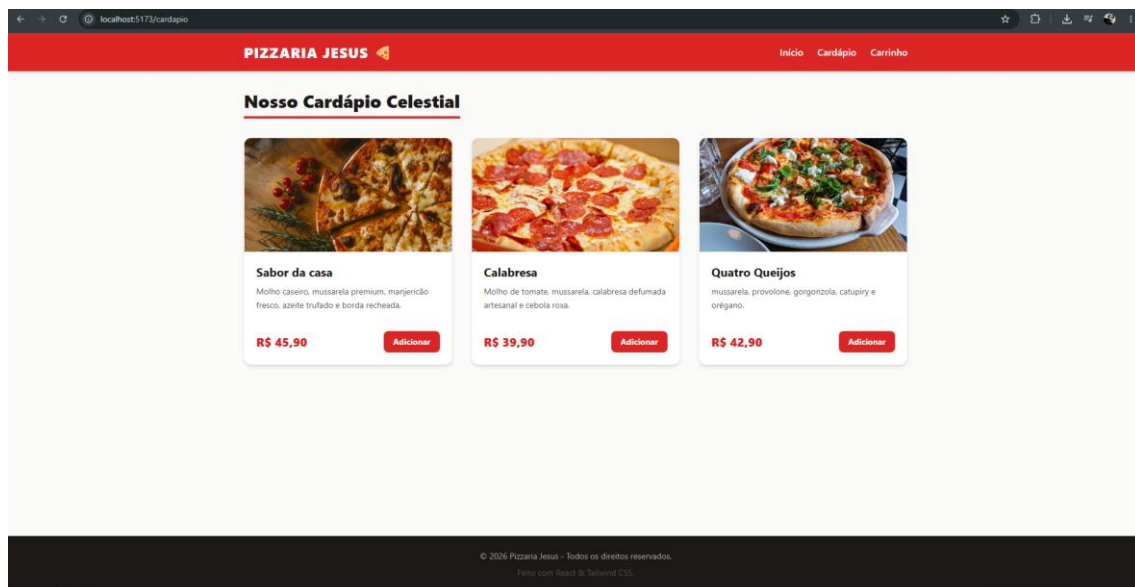


Figura 2: Página de Gerenciamento do Carrinho de Compras

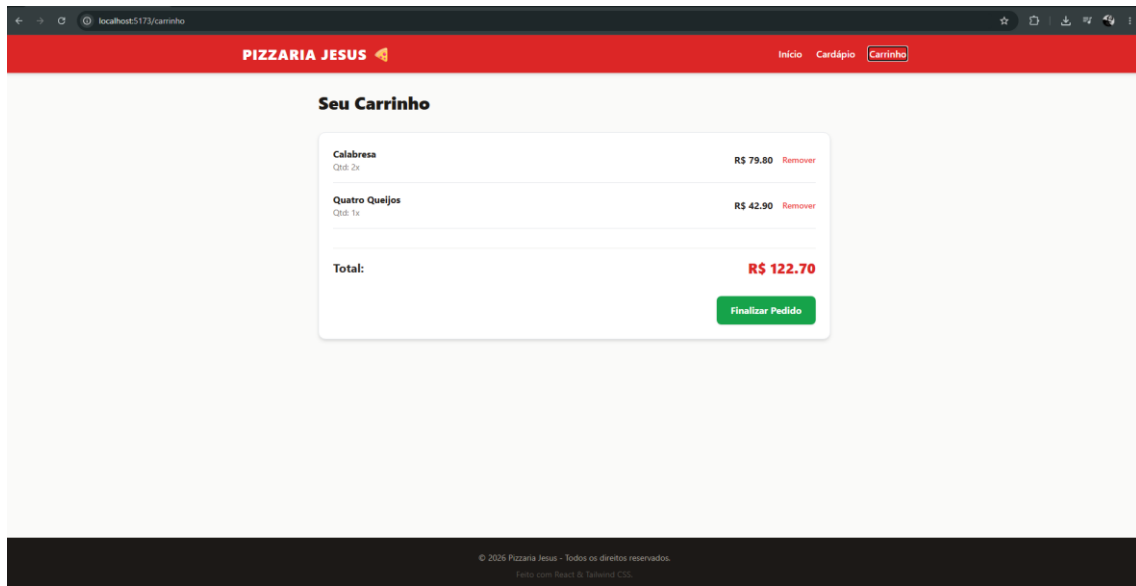


Figura 3: Formulário de Finalização de Pedido (Checkout) com Validação

